

Smart Ceiling Fan Control System

Computer Networks (EE427) Mini Project

Submitted By G1

Aayush Anand (231EE101)
Adarsh Akshat (231EE102)
Ahan Halder (231EE104)
Anagha Hatwar (231EE106)

B.Tech (III), Electrical and Electronics Engineering

Submitted to

Prof. Panduranga Vittal K
Dept. of Electrical and Electronics Engineering



National Institute of Technology Karnataka,
Surathkal

April 6, 2026

Contents

Abstract	3
1 Introduction	3
2 Objectives	4
3 Hardware Design and Circuit Description	4
3.1 Bill of Materials	4
3.2 Motor Drive Sub-Circuit	5
3.3 LED Indicator Sub-Circuit	6
3.4 Power Supply Architecture	6
3.5 Wokwi Simulation	6
3.6 Physical Build	7
4 Methodology of Implementation	7
4.1 System Architecture and Layers	7
4.2 Speed State Mapping	8
4.3 ThingSpeak Channel and Field Mapping	9
4.4 Google Assistant Voice Command Integration	9
4.4.1 Setup Steps for Google Assistant Integration	9
4.5 Function Schematic	10
4.6 Execution Flow Diagram	11
4.7 Firmware Architecture	12
4.8 Code Functionality Overview	12
4.8.1 System Initialization	12
4.8.2 LED Status Indication	13
4.8.3 Motor Speed Control	14
4.8.4 Unified Speed Update Routine	14
4.8.5 Web Dashboard Control	15
4.8.6 SinricPro Voice and App Control	15
4.8.7 ThingSpeak Synchronization	15
4.8.8 Main Loop	16
5 Proof of Execution and Results	17
5.1 Browser Dashboard	17
5.2 SinricPro and Google Assistant Dashboard	18
5.3 ThingSpeak Dashboard	19
5.4 Serial Monitor Output	20

5.5	Verification Checklist	21
6	Wireshark Network Packet Analysis	21
6.1	DHCP — Dynamic Host Configuration Protocol	22
6.1.1	Protocol Background	22
6.1.2	Observed Exchange	22
6.1.3	Relevance to the Project	23
6.2	DNS — Domain Name System	23
6.2.1	Protocol Background	23
6.2.2	Observed Exchange	23
6.2.3	Relevance to the Project	24
6.3	IGMPv2 — Internet Group Management Protocol	24
6.3.1	Protocol Background	24
6.3.2	Observed Exchange	25
6.3.3	Relevance to the Project	25
6.4	TCP — Transmission Control Protocol	26
6.4.1	Protocol Background	26
6.4.2	Observed Exchange	26
6.4.3	Relevance to the Project	27
6.5	HTTP — HyperText Transfer Protocol	28
6.5.1	Protocol Background	28
6.5.2	Observed Exchange	28
6.5.3	Why HTTP and not HTTPS for ThingSpeak?	29
6.6	TLSv1.2 — Transport Layer Security	29
6.6.1	Protocol Background	30
6.6.2	Observed Frames	30
6.6.3	Why TLS Matters for This Project	31
6.7	Protocol Summary and OSI Layer Mapping	31
7	Conclusion	32

Abstract

This report presents the complete design, simulation, and physical implementation of an Internet of Things (IoT) based smart ceiling fan speed control system. The system is built around an **ESP32 WROOM** microcontroller and integrates two independent cloud platforms — **ThingSpeak** for data logging and command transport, and **SinricPro** for smart-device ecosystem connectivity — alongside a **Google Assistant** voice-command interface. Four discrete fan speed states (OFF, LOW, MEDIUM, HIGH) are implemented using hardware PWM to drive a real 5 V DC toy motor through an **IRLZ44N N-channel MOSFET**, with a **1N4007** flyback diode providing back-EMF protection. Three green LEDs serve as concurrent visual speed indicators.

A browser-based HTML/CSS/JS dashboard allows users to issue speed commands remotely through ThingSpeak. SinricPro, linked to Google Assistant, enables natural-language voice commands such as “Set Ceiling Fan speed 3”. The system was validated using Wokwi circuit simulation prior to hardware assembly. Network behavior was analyzed using **Wireshark** packet captures, revealing the full protocol stack in operation: DHCP for IP assignment, DNS for hostname resolution, IGMPv2 for multicast group membership, TCP for reliable transport, HTTP for ThingSpeak API calls, and TLSv1.2 for encrypted SinricPro communication. This report details the complete methodology, hardware circuit, firmware architecture, cloud integration, voice-command setup, execution flow, photographic proof of execution, and a thorough Wireshark protocol analysis.

1 Introduction

Smart home appliances represent a core application domain of the Internet of Things. Remotely controlling household devices reduces energy consumption, improves convenience, and provides real-time operational data. A ceiling fan is an ideal candidate for an end-to-end IoT demonstration because it has simple, discrete operating states and a direct relationship between the control signal and physical output.

In this project, a **complete IoT pipeline** is demonstrated across multiple layers. A command originates at either a browser dashboard, the SinricPro mobile application, or a Google Assistant voice query; travels through cloud services; is received and acted upon by an embedded ESP32 microcontroller; and results in physical actuation of a DC motor via a MOSFET driver circuit. The ESP32 simultaneously reports the resulting state back to ThingSpeak, completing a closed feedback loop visible in cloud dashboards.

The hardware uses a low-voltage 5 V DC toy motor instead of a mains-connected AC motor, eliminating high-voltage hazards while fully demonstrating the software and control architecture. Network traffic generated by the ESP32 was captured and analysed

with Wireshark, providing direct insight into the protocols underpinning the system and connecting the hardware implementation to the Computer Networks theory studied in EE427.

2 Objectives

1. Design and physically build an IoT-based ceiling fan speed control system using the ESP32 WROOM microcontroller.
2. Implement four discrete fan speed levels — OFF (0%), LOW (33%), MEDIUM (67%), and HIGH (100% PWM duty cycle) — using the ESP32's hardware LEDC peripheral.
3. Drive a 5 V DC toy motor through an IRLZ44N N-channel MOSFET, with a 1N4007 flyback diode for back-EMF protection.
4. Use three green LEDs as a cumulative visual speed level indicator.
5. Implement a responsive browser-based HTML/CSS/JS dashboard for remote speed control.
6. Integrate ThingSpeak as the cloud command transport and state logging channel.
7. Integrate SinricPro as the smart-device ecosystem and voice-assistant bridge.
8. Link SinricPro to Google Assistant to enable natural-language voice commands for all four speed states.
9. Validate circuit correctness using Wokwi simulation prior to hardware assembly.
10. Capture and analyse ESP32 network traffic using Wireshark, identifying DHCP, DNS, IGMPv2, HTTP, TCP, and TLSv1.2 protocol activity.

3 Hardware Design and Circuit Description

3.1 Bill of Materials

Table 1 lists all hardware components used in the physical build.

Table 1: Bill of Materials

S.N.	Component	Qty	Role in Circuit
1	ESP32 WROOM ESP-32S (2.4 GHz Dual-Mode Wi-Fi + Bluetooth, Dual Core)	1	Main microcontroller; runs firmware, PWM generation, Wi-Fi communication
2	5 V DC Toy Motor	1	Physical actuator representing the ceiling fan load
3	IRLZ44N N-Channel Logic-Level MOSFET	1	Switches motor power under ESP32 PWM control
4	1N4007 Rectifier Diode	1	Flyback protection for MOSFET against motor back-EMF
5	Green LED	3	Visual speed level indicators (OFF/LOW/MEDIUM/HIGH)
6	220 Ω Resistor	1	Current limiter in motor gate drive circuit
7	3.3 Ω Resistor	3	Current limiters for the three indicator LEDs
8	USB Breakout Cable	1	5 V power supply for motor circuit from USB port
9	Breadboard	1	Prototyping platform
10	Jumper Wires	Several	Interconnections between components

3.2 Motor Drive Sub-Circuit

The 5 V DC motor is connected between the USB breakout 5 V rail and the **drain** of the IRLZ44N MOSFET. The MOSFET source connects to ground. The gate is driven by a PWM-capable ESP32 GPIO pin through a 220 Ω gate resistor. This resistor limits gate-charge current spikes and damps ringing on the gate line.

The IRLZ44N is a **logic-level MOSFET**: it achieves full enhancement at $V_{GS} \approx 2.8$ –3.3 V, making it directly compatible with the ESP32’s 3.3 V GPIO outputs without a level shifter. Its on-resistance ($R_{DS(on)} \approx 22 \text{ m}\Omega$) results in negligible conduction loss at the motor currents involved.

The **1N4007 diode** is placed anti-parallel across the motor terminals (cathode to 5 V, anode to drain). When the MOSFET turns off, the motor’s inductance forces current to continue flowing; the diode provides a safe recirculation path, clamping the drain voltage spike and preventing MOSFET avalanche breakdown.

3.3 LED Indicator Sub-Circuit

Three green LEDs are connected between dedicated ESP32 GPIO pins and ground, each with a 3.3Ω current-limiting resistor. The LEDs display the speed level cumulatively (Table 2). The ESP32 pin current limit ($\leq 40\text{ mA}$) ensures safe operation.

3.4 Power Supply Architecture

The ESP32 is powered from a computer USB port via its on-board micro-USB connector (3.3 V regulated internally). The motor circuit draws its 5 V supply from a **separate USB breakout cable** connected to the same USB source, ensuring motor inrush current does not flow through the ESP32's internal LDO regulator.

3.5 Wokwi Simulation

Before physical assembly, the complete circuit was simulated in **Wokwi** to verify GPIO pin assignments, LED behaviour for each speed state, PWM waveform correctness, and firmware logic—all without hardware risk.

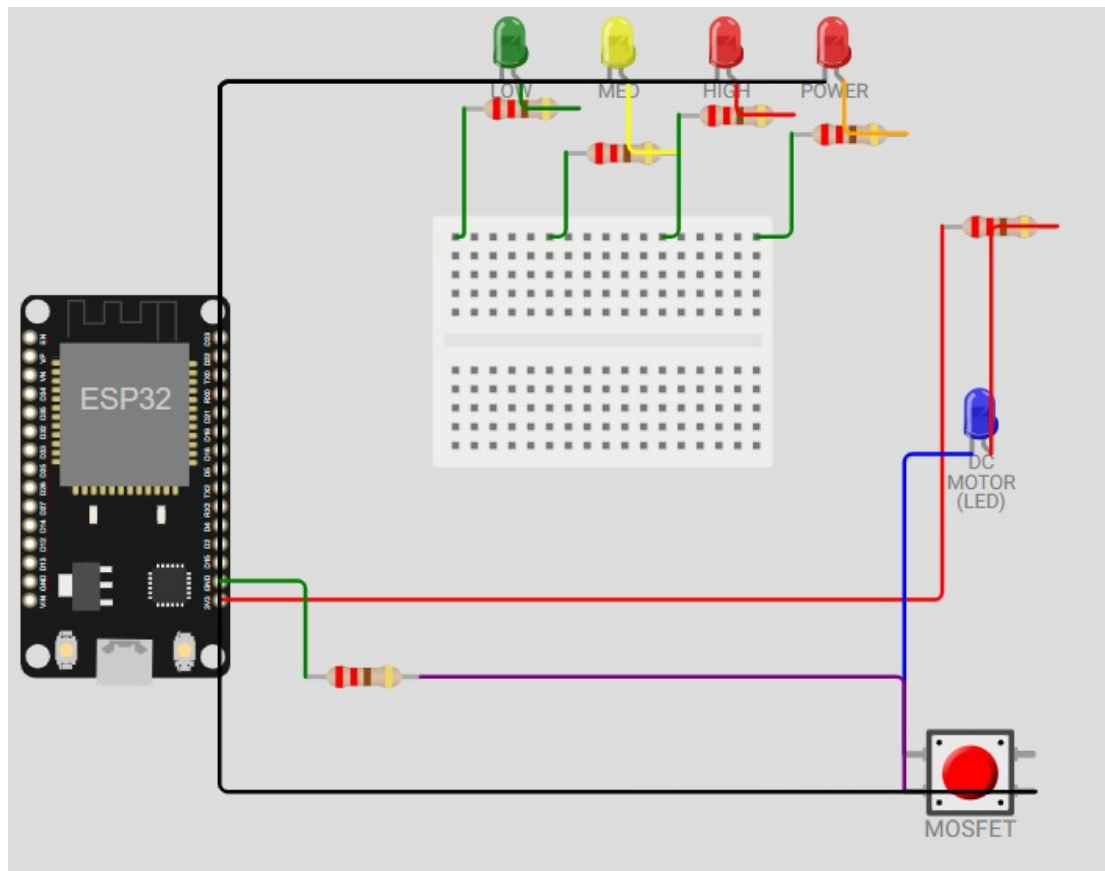
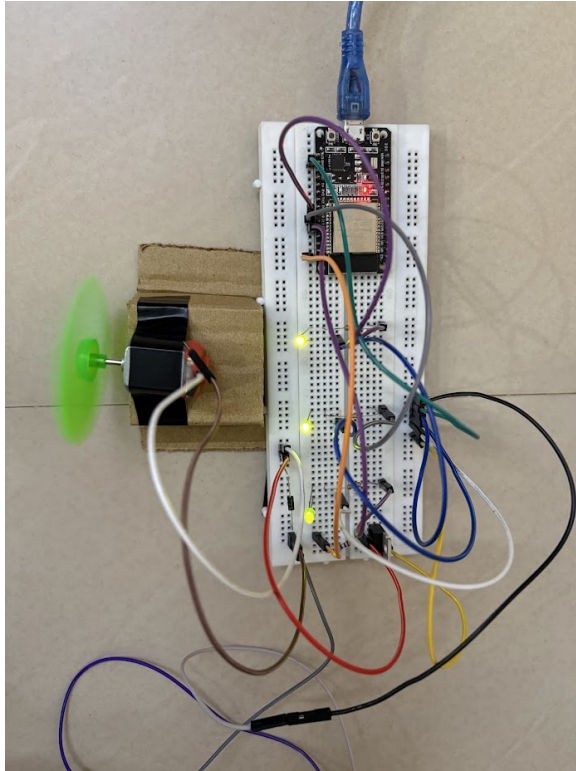
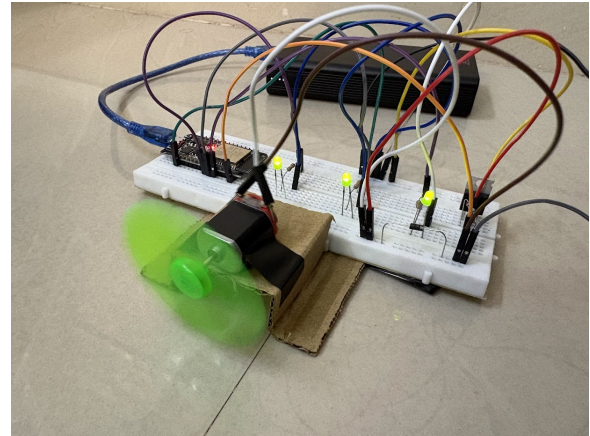


Figure 1: Wokwi online circuit simulation of the ceiling fan control system, showing the ESP32 WROOM board, three indicator LEDs, MOSFET, diode, and motor connections prior to physical build.

3.6 Physical Build



(a) Top view of assembled circuit.



(b) Side view of assembled circuit.

Figure 2: Physical hardware assembly on breadboard. The IRLZ44N MOSFET, 1N4007 diode, three green LEDs, USB breakout cable, and 5 V toy motor are all visible.

4 Methodology of Implementation

4.1 System Architecture and Layers

The system is partitioned into five clearly separated layers. This separation ensures modularity: each layer can be tested independently, and failures are localised.

Layer 1 — User Interface Layer:

Three parallel interfaces exist: (a) a browser-based HTML/CSS/JS dashboard with four speed buttons, (b) the SinricPro mobile application, and (c) Google Assistant via a SinricPro–Google Home linkage. All three converge on the same control logic in the ESP32 firmware.

Layer 2 — Cloud Communication Layer:

ThingSpeak (channel 3322916) stores speed commands in Field 3 and receives state updates in Fields 1 and 2. SinricPro’s cloud servers maintain a persistent WebSocket connection with the ESP32 for low-latency event

delivery.

Layer 3 — Embedded Control Layer:

The ESP32 firmware connects to Wi-Fi, subscribes to SinricPro events, polls ThingSpeak for browser-originated commands, and calls the central `applyFanSpeed()` function regardless of which control path is active.

Layer 4 — Actuation Layer:

The MOSFET gate is driven at a PWM duty cycle corresponding to the selected speed level, modulating motor speed. Three LED GPIO pins are simultaneously set in a cumulative pattern.

Layer 5 — Remote Integration Layer:

SinricPro bridges the ESP32 to the Google Home ecosystem, enabling Google Assistant voice commands to trigger the same firmware callback as any other speed-change event.

4.2 Speed State Mapping

Table 2: Speed state mapping: command code, PWM duty cycle, LED outputs, and voice command phrase

Code	Label	PWM Duty	Motor	LED1	LED2	LED3
0	OFF	0%	Stopped	OFF	OFF	OFF
1	LOW	33%	Slow	ON	OFF	OFF
2	MEDIUM	67%	Medium	ON	ON	OFF
3	HIGH	100%	Full	ON	ON	ON

4.3 ThingSpeak Channel and Field Mapping

Table 3: ThingSpeak field roles in the project

Field	Written By	Direction	Purpose
Field 1	ESP32 firmware	ESP32 → Cloud	Numeric speed value (0–3); read by dashboard on page load.
Field 2	ESP32 firmware	ESP32 → Cloud	Human-readable speed label (OFF / LOW / MEDIUM / HIGH).
Field 3	Browser / user	Browser → Cloud	Speed command (0–3) consumed by ESP32 on next poll.

4.4 Google Assistant Voice Command Integration

The voice-command capability was added by linking the SinricPro account to **Google Home** via the official SinricPro Google Home action. Once linked, Google Assistant can address the device by its SinricPro display name (*Ceiling Fan*) and issue commands that are relayed through the SinricPro cloud to the ESP32 in real time over the existing WebSocket connection. The four supported voice commands and their effects are listed in Table 4.

Table 4: Google Assistant voice commands and corresponding speed codes

Voice Command (Google Assistant)	Code	Effect
“Hey Google, turn off Ceiling Fan”	0	Motor stops, all LEDs OFF
“Hey Google, set Ceiling Fan speed 1”	1	PWM 33%, LED1 ON
“Hey Google, set Ceiling Fan speed 2”	2	PWM 67%, LED1+2 ON
“Hey Google, set Ceiling Fan speed 3”	3	PWM 100%, all LEDs ON

4.4.1 Setup Steps for Google Assistant Integration

Step 1. Create a SinricPro account and register a *Fan* device. Note the App Key, App Secret, and Device ID.

Step 2. Flash the ESP32 firmware with these credentials. Confirm the device appears *online* in the SinricPro dashboard.

Step 3. Open the **Google Home** app on a smartphone. Tap “+” → *Set up device* → *Works with Google*.

Step 4. Search for **SinricPro** in the list of services and tap to link. Log in with SinricPro credentials when prompted.

Step 5. Google Home imports the *Ceiling Fan* device. Assign it to a room (e.g., “Bedroom”).

Step 6. Verify by saying “Hey Google, set Ceiling Fan speed 2” — the motor should immediately step to 67% PWM duty cycle.

The voice command travels from the Google Assistant cloud to the SinricPro cloud over HTTPS, then from SinricPro to the ESP32 over the established TLS-encrypted WebSocket. The ESP32 decodes the fan-speed payload and calls `applyFanSpeed()` with the corresponding code. The entire path from voice utterance to motor speed change completes in approximately 1–2 seconds.

4.5 Function Schematic

Figure 3 presents the complete information-flow schematic, showing all control paths — browser, SinricPro app, and Google Assistant — converging on the ESP32.

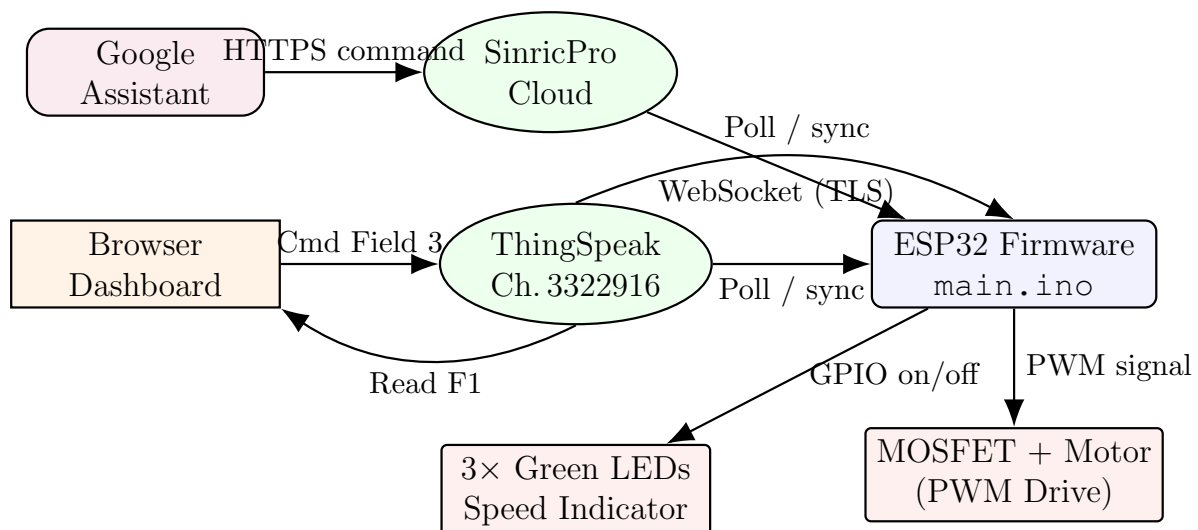


Figure 3: Complete function schematic showing all three control paths (Browser, SinricPro app, Google Assistant) converging on the ESP32, and both cloud services in their correct roles.

4.6 Execution Flow Diagram

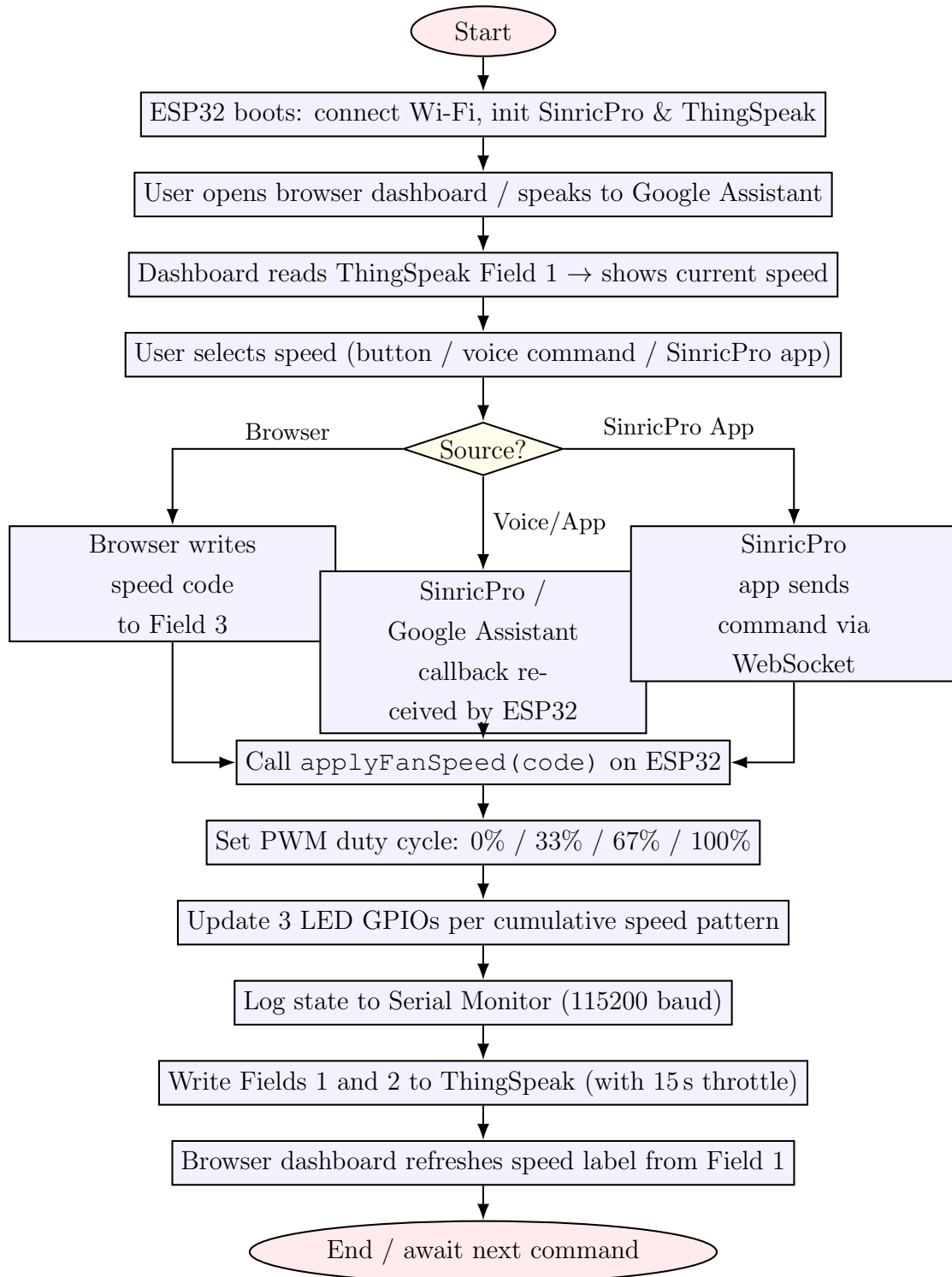


Figure 4: Detailed execution flow showing all three control paths (browser, SinricPro app, Google Assistant) merging into a single `applyFanSpeed()` call on the ESP32, followed by motor control and cloud state update.

4.7 Firmware Architecture

The firmware centres on a single function: `applyFanSpeed(int speed)`. This is the **single point of truth** for all speed changes. Regardless of whether the command arrives from ThingSpeak polling or a SinricPro/Google Assistant callback, this function is called with the decoded integer (0–3).

PWM Configuration: The ESP32 LEDC peripheral is configured at 1 kHz with 8-bit resolution. The four speed levels map to LEDC duty values of 0, 85, 170, and 255 (corresponding to 0%, 33%, 67%, and 100% duty cycle respectively).

ThingSpeak Throttling: A software timer ensures ThingSpeak is written at most once every 15 seconds, respecting the free-tier rate limit. The browser dashboard mirrors this with a 16-second cooldown progress bar.

SinricPro Callback: The `onFanSpeedChange()` callback registered with the SinricPro library decodes the incoming speed level and immediately calls `applyFanSpeed()`, producing a sub-2-second end-to-end latency for voice and app commands.

4.8 Code Functionality Overview

The firmware is structured around a single fan-speed state variable, and all control sources update the same state so that the LEDs, motor output, and cloud platforms remain synchronized. :contentReference[oaicite:0]index=0

4.8.1 System Initialization

The `setup()` function configures the LED pins, initializes PWM for the motor, connects the ESP32 to Wi-Fi, starts the web server, and enables SinricPro support. This is the point where the complete hardware and network stack is brought online. :contentReference[oaicite:1]index=1

```
void setup() {
  Serial.begin(115200);
  delay(200);

  pinMode(POWER_LED, OUTPUT);
  pinMode(LED_LOW, OUTPUT);
  pinMode(LED_MED, OUTPUT);
  pinMode(LED_HIGH, OUTPUT);
  runLEDStartupTest();
  updateLEDs();

  ledcAttach(MOTOR_PIN, PWM_FREQ, PWM_RES);
  ledcWrite(MOTOR_PIN, 0);
```

```
WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print('.');
}

server.on("/", handleRoot);
server.on("/set", handleSet);
server.begin();

SinricProFanUS &fan = SinricPro[FAN_DEVICE_ID];
fan.onPowerState(onPowerState);
fan.onRangeValue(onRangeValue);
SinricPro.begin(SINRIC_APP_KEY, SINRIC_APP_SECRET);

queueThingSpeakSync();
}
```

4.8.2 LED Status Indication

The `updateLEDs()` function provides direct visual feedback for the current fan speed. The POWER LED indicates whether the fan is active, while the LOW, MEDIUM, and HIGH LEDs represent the selected speed level. :contentReference[oaicite:2]index=2

```
void updateLEDs() {
    digitalWrite(POWER_LED, fanSpeed > 0 ? LOW : HIGH);

    switch (fanSpeed) {
        case 0:
            digitalWrite(LED_LOW, LOW);
            digitalWrite(LED_MED, LOW);
            digitalWrite(LED_HIGH, LOW);
            break;
        case 1:
            digitalWrite(LED_LOW, HIGH);
            digitalWrite(LED_MED, LOW);
            digitalWrite(LED_HIGH, LOW);
            break;
        case 2:
            digitalWrite(LED_LOW, HIGH);
            digitalWrite(LED_MED, HIGH);
```

```

        digitalWrite(LED_HIGH, LOW);
        break;
    case 3:
        digitalWrite(LED_LOW, HIGH);
        digitalWrite(LED_MED, HIGH);
        digitalWrite(LED_HIGH, HIGH);
        break;
    }
}

```

4.8.3 Motor Speed Control

The `updateMotor()` function drives the fan motor using PWM. Each fan level maps to a fixed duty cycle, which controls the effective motor output. :contentReference[oaicite:3]index=3

```

void updateMotor() {
    ledcWrite(MOTOR_PIN, pwmDuty[fanSpeed]);

    Serial.println(F("_[MOTOR_]"));
    Serial.printf ("_PWM_Duty_:_%d/_255\n", pwmDuty[fanSpeed]);
    Serial.printf ("_Est._Voltage:_.2f_V_(at_5V_supply)\n",
        pwmDuty[fanSpeed] / 255.0f * 5.0f);
}

```

4.8.4 Unified Speed Update Routine

The `applyFanSpeed()` function is the main control path used by every input source. It constrains the speed value, updates the internal state, refreshes the LEDs and motor, and marks the state for ThingSpeak upload. :contentReference[oaicite:4]index=4

```

void applyFanSpeed(int spd) {
    spd = constrain(spd, 0, 3);
    if (spd == fanSpeed) return;

    fanSpeed = spd;
    updateLEDs();
    updateMotor();

    queueThingSpeakSync();
}

```

4.8.5 Web Dashboard Control

The embedded web server serves a browser-based dashboard, and the `handleSet()` route accepts speed requests from the page. This allows local control through a phone or laptop on the same network. :contentReference[oaicite:5]index=5

```
void handleSet() {
    if (server.hasArg("speed")) {
        int requested = server.arg("speed").toInt();
        applyFanSpeed(requested);
    }
    server.send(200, "text/plain", "OK");
}
```

4.8.6 SinricPro Voice and App Control

The SinricPro callbacks allow the fan to respond to app-based and voice-assistant commands. Power commands map to ON/OFF behavior, while range commands map directly to the fan speed levels. :contentReference[oaicite:6]index=6

```
bool onPowerState(const String& id, bool& on) {
    applyFanSpeed(on ? 1 : 0);
    return true;
}

bool onRangeValue(const String& id, int& val) {
    applyFanSpeed(val);
    val = fanSpeed;
    return true;
}
```

4.8.7 ThingSpeak Synchronization

The firmware uploads the current fan state to ThingSpeak and also polls for remote command updates. This makes the system usable for monitoring and control from the cloud. :contentReference[oaicite:7]index=7

```
void mirrorThingSpeakIfNeeded() {
    if (!tsDirty) return;
    if (millis() - lastTSWrite < 16000UL) return;

    HTTPClient http;
    String url = "http://api.thingspeak.com/update?api_key=";
```



```
url += TS_WRITE_API;
url += "&field1=" + String(fanSpeed);
url += "&field2=" + String(speedNames[fanSpeed]);

http.begin(url);
int code = http.GET();
String body = http.getString();
http.end();
}
```

```
void pollThingSpeakCommand() {
    if (millis() - lastPoll < 16000UL) return;
    lastPoll = millis();

    HTTPClient http;
    String url = "http://api.thingspeak.com/channels/";
    url += TS_CHANNEL_ID;
    url += "/fields/3/last.json?api_key=";
    url += TS_READ_API;

    http.begin(url);
    int code = http.GET();
    http.end();
}
```

4.8.8 Main Loop

The `loop()` function continuously services the web server, SinricPro events, ThingSpeak upload, and ThingSpeak polling. This keeps the fan responsive to all control sources at the same time. [:contentReference\[oaicite:8\]index=8](#)

```
void loop() {
    server.handleClient();
    SinricPro.handle();
    mirrorThingSpeakIfNeeded();
    pollThingSpeakCommand();
}
```

5 Proof of Execution and Results

5.1 Browser Dashboard

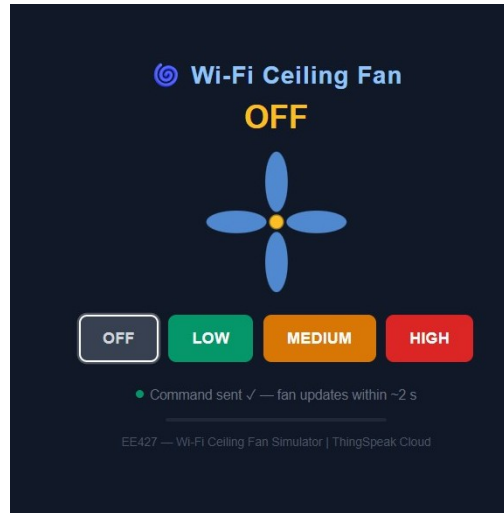


Figure 5: Browser dashboard showing the four speed control buttons, animated fan icon, current speed label, ThingSpeak write cooldown bar, and status feedback line.

The dashboard reads ThingSpeak Field 1 on page load to display the last known speed. Pressing a speed button writes the corresponding code to Field 3, triggers the fan animation at proportional rotation speed, and starts the 16-second cooldown bar. The status line confirms whether the ThingSpeak write returned HTTP 200.

5.2 SinricPro and Google Assistant Dashboard

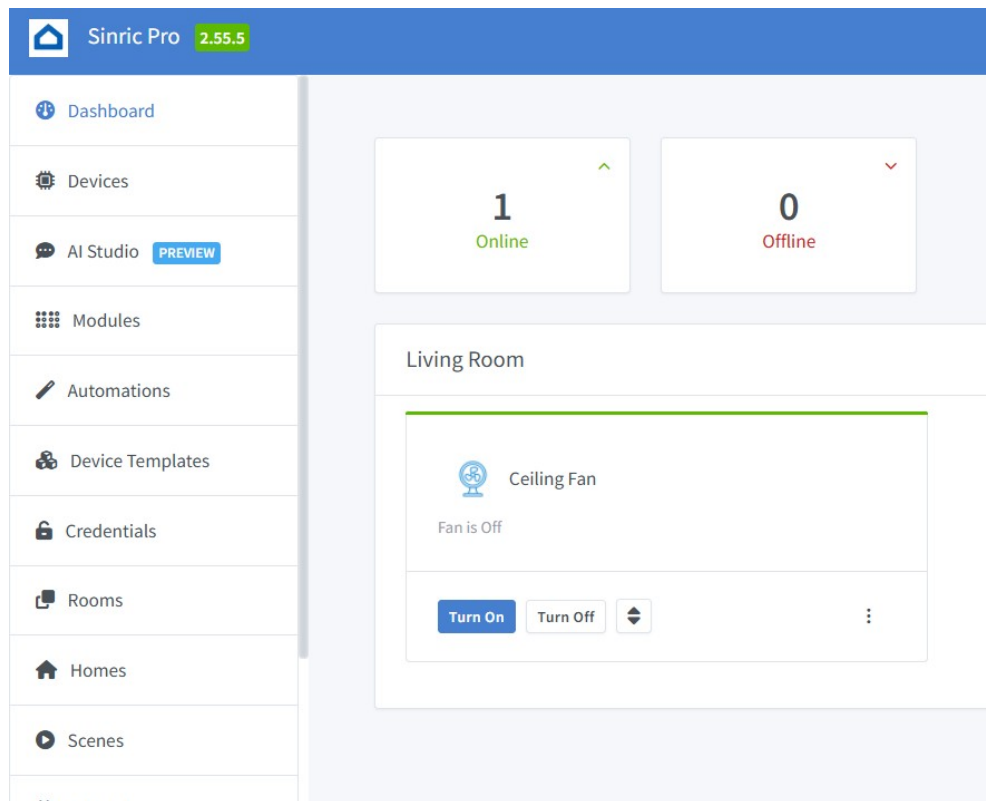


Figure 6: SinricPro cloud dashboard showing the registered *Ceiling Fan* device. The device is online and linked to the Google Home account, enabling the four Google Assistant voice commands.

The SinricPro dashboard confirms that: (a) the *Ceiling Fan* device is registered and online; (b) the device template exposes fan speed levels; and (c) the Google Home service link is active, enabling Google Assistant commands. Commands issued via voice travel from the Google Assistant cloud to SinricPro servers over HTTPS, then to the ESP32 over the existing TLS WebSocket connection, with observed end-to-end latency of 1–2 seconds.

5.3 ThingSpeak Dashboard

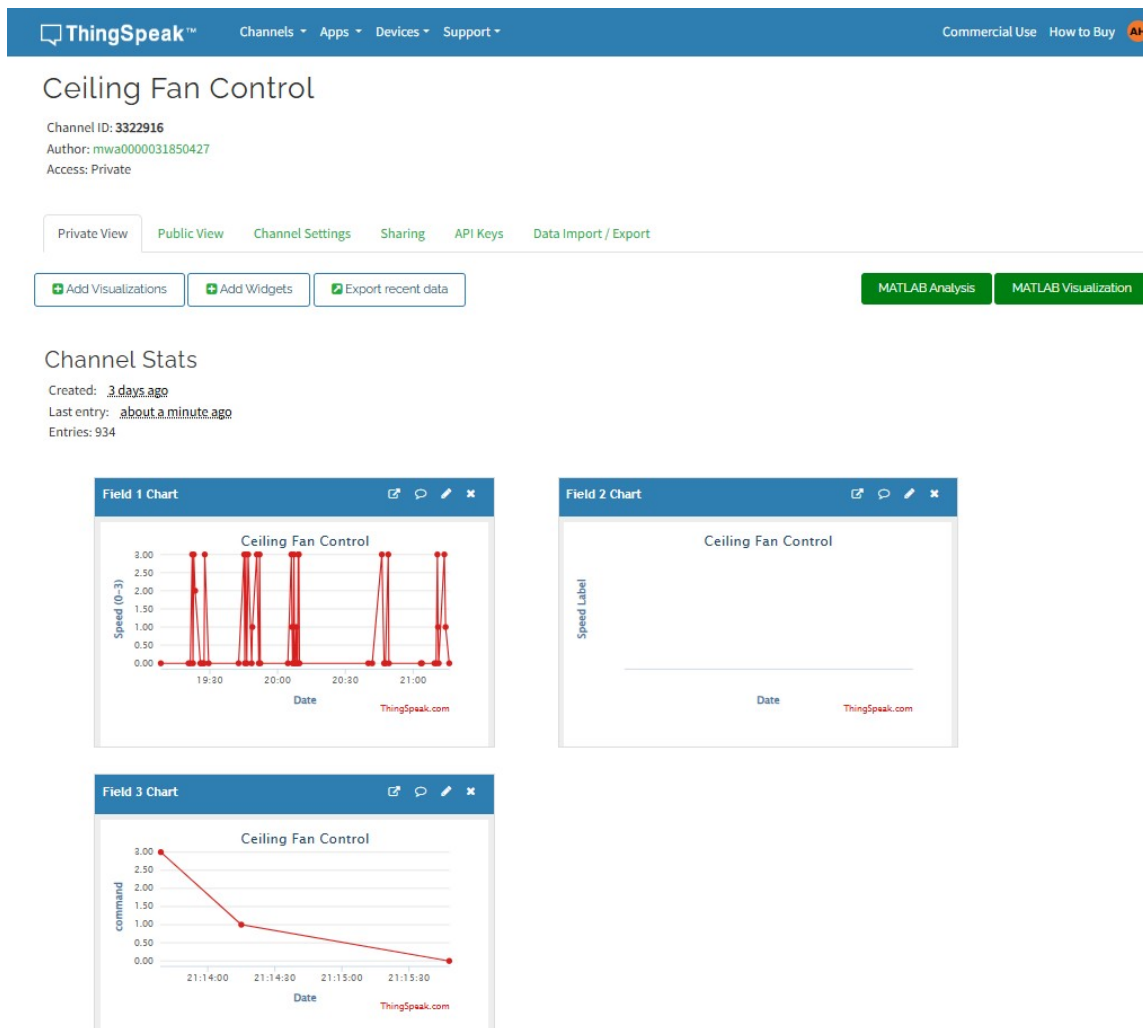


Figure 7: ThingSpeak channel dashboard for *Ceiling Fan Control* (Ch. 3322916). Field 1 plots show multiple speed transitions; Field 2 shows the corresponding labels; Field 3 shows browser-originated command values.

The historical data plots on the ThingSpeak dashboard confirm: Field 1 (speed value 0–3) updates after each ESP32 write; Field 2 (speed label) transitions between OFF, LOW, MEDIUM, and HIGH; Field 3 (command) shows values written by the browser UI. Multiple speed transitions are evident in the time-series charts, confirming end-to-end operation across multiple test cycles.

5.4 Serial Monitor Output

```

=====
|   Wi-Fi Ceiling Fan   |   EE427       |
=====
LEDs      : Initialized
Motor PWM : Initialized (duty = 0, motor OFF)

WiFi      : Connecting to "Ahan_iPhone" ... Connected!
IP Address : 172.20.10.13
Web Server : Running on port 80
SinricPro  : Connected

=====
System is READY. Waiting for commands...
=====

[ THINGSPEAK ] Uploading current fan state...
Status      : OK (HTTP 200)
Entry #     : 925
Uploaded    : Speed = OFF (0)

[ THINGSPEAK ] Remote command received!
Command     : Set speed to Level 0 (OFF)

[ WEB ] Speed change via dashboard
Requested   : Level 3 (HIGH)
[ MOTOR ]
PWM Duty    : 166 / 255
Est. Voltage: 3.25 V (at 5V supply)

-----
[ FAN ] Speed Changed
-----
New Speed   : HIGH (Level 3)
Speed Bar   : [###]
-----

```

Figure 8: Arduino Serial Monitor at 115200 baud, showing Wi-Fi connection, SinricPro handshake, ThingSpeak Field 3 polls, `applyFanSpeed()` calls with GPIO states, and successful ThingSpeak state write responses (HTTP 200).

The serial log is the ground-truth trace of firmware execution. Key events visible include: Wi-Fi IP assignment; SinricPro WebSocket connection; ThingSpeak Field 3 read returning the command code; the `applyFanSpeed()` call specifying PWM duty and LED states; and ThingSpeak write responses confirming HTTP 200 success. Every speed transition is logged with a millisecond timestamp, confirming correct sequential operation.

5.5 Verification Checklist

Table 5: Verification checklist linking each system objective to its evidence

Functionality	Figure	Status
Browser dashboard loads and shows current speed	5	Confirmed
Speed buttons write to ThingSpeak Field 3	7	Confirmed
ESP32 connects to Wi-Fi and SinricPro	8	Confirmed
ESP32 polls ThingSpeak Field 3 for commands	8	Confirmed
PWM duty changes per speed level	1, hardware	Confirmed
LEDs indicate speed cumulatively	2	Confirmed
Motor spins at varying speed for each level	Physical demo	Confirmed
State mirrored to ThingSpeak Fields 1 & 2	7	Confirmed
SinricPro device registered and online	6	Confirmed
Google Assistant voice commands functional	6, Table 4	Confirmed
Wokwi simulation validated pre-build	1	Confirmed

6 Wireshark Network Packet Analysis

Wireshark was used to capture all network traffic generated by the ESP32 (IP address: 192.168.137.180, MAC address: e2:e9:aa:77:07:97 — an Espressif locally administered address) during a complete operational cycle. The capture filter `ip.addr == 192.168.137.180` was applied to isolate ESP32 traffic. Six protocol categories were identified and are analysed below, each corresponding to one Wireshark capture screenshot.

OSI Layer Reference: The observed protocols span all layers — DHCP and ARP at Layer 2/3, DNS at Layer 7 (over UDP/Layer 4), IGMPv2 at Layer 3, TCP at Layer 4, HTTP at Layer 7, and TLS at Layer 5/6. Together they illustrate the complete network stack activated when an embedded IoT device connects to the internet.

6.1 DHCP — Dynamic Host Configuration Protocol

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.137.4	192.168.137.4	TLSv1.2	53	Application Data
2	0.044566800	192.168.137.4	162.55.80.75	TCP	54	54639 → 443 [ACK] Seq=1 Ack=32 Win=5094 Len=0
3	0.044566800	192.168.137.4	162.55.80.75	TLSv1.2	89	Application Data
4	0.374642800	162.55.80.75	192.168.137.4	TCP	54	443 → 54639 [ACK] Seq=32 Ack=36 Win=42050 Len=0
6	1.937818600	192.168.137.1	192.168.137.4	DHCP	344	DHCP Offer - Transaction ID 0x96676d93
8	1.956640400	192.168.137.1	192.168.137.4	DHCP	344	DHCP ACK - Transaction ID 0x96676d93
12	3.359860500	192.168.137.4	224.9.9.9	IGMPv2	46	Membership Report group 224.9.9.9
15	3.364456100	192.168.137.4	192.168.137.1	DNS	73	Standard query 0xb707 A ws.sinric.pro
16	3.448338000	192.168.137.1	192.168.137.4	DNS	102	Standard query response 0xb707 A ws.sinric.pro A 162.55.80.75
17	3.526766500	192.168.137.4	162.55.80.75	TCP	62	62281 → 443 [SYN] Seq=0 Win=5760 Len=0 MSS=1436 SACK_PERM
18	3.558858600	192.168.137.4	224.9.9.9	IGMPv2	46	Membership Report group 224.9.9.9
20	3.896248200	162.55.80.75	192.168.137.4	TCP	62	443 → 62281 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1340 SACK_PERM
21	4.336278300	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=1 Ack=1 Win=5760 Len=0
22	4.339246900	192.168.137.4	162.55.80.75	TLSv1.2	344	Client Hello (SNI=ws.sinric.pro)
23	4.681272700	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=1 Ack=291 Win=42050 Len=0
24	4.682744600	162.55.80.75	192.168.137.4	TLSv1.2	1394	Server Hello
25	4.683423200	162.55.80.75	192.168.137.4	TCP	1394	443 → 62281 [PSH, ACK] Seq=1341 Ack=291 Win=42050 Len=1340 [TCP PDU reassembled in 26]
26	4.689079500	162.55.80.75	192.168.137.4	TLSv1.2	370	Certificate, Server Key Exchange, Server Hello Done
28	4.755767800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2681 Win=3080 Len=0
29	4.757564800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2997 Win=5446 Len=0
30	5.032174400	192.168.137.4	162.55.80.75	TLSv1.2	96	Client Key Exchange
31	5.038832500	192.168.137.4	162.55.80.75	TLSv1.2	60	Change Cipher Spec
32	5.040909600	192.168.137.4	162.55.80.75	TLSv1.2	99	Encrypted Handshake Message


```

Frame 1: Packet, 85 bytes on wire (680 bits), 85 bytes captured (680 bits) on interface \Device\NPF...
Ethernet II, Src: e2:e9:aa:77:07:97 (e2:e9:aa:77:07:97), Dst: Espressif_e3:2c:ec (48:9d:31:e3:2c:ec)
  Destination: Espressif_e3:2c:ec (48:9d:31:e3:2c:ec)
    ....0. .... = LG bit: Globally unique address (factory default)
    ....0. .... = IG bit: Individual address (unicast)
  Source: e2:e9:aa:77:07:97 (e2:e9:aa:77:07:97)
    ....1. .... = LG bit: Locally administered address (this is NOT the factory default)
    ....0. .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
    [Stream index: 0]
  Internet Protocol Version 4, Src: 162.55.80.75, Dst: 192.168.137.4
  Transmission Control Protocol, Src Port: 443, Dst Port: 54639, Seq: 1, Ack: 1, Len: 31
  Transport Layer Security
    [Stream index: 0]
    TLSv1.2 Record Layer: Application Data Protocol: Hypertext Transfer Protocol

```

Figure 9: Wireshark capture of the DHCP four-way handshake. The ESP32 broadcasts a DHCP Discover, receives an Offer, sends a Request, and receives an Acknowledgement assigning it IP 192.168.137.180.

6.1.1 Protocol Background

DHCP (Dynamic Host Configuration Protocol, RFC 2131) automates the assignment of IP addresses, subnet masks, default gateways, and DNS server addresses to network hosts. It operates over **UDP**, using port 67 (server) and port 68 (client), at the **Application Layer** of the TCP/IP model.

6.1.2 Observed Exchange

When the ESP32 boots and its Wi-Fi interface comes up, it has no IP address. The DORA sequence unfolds as follows:

- DHCP Discover** — The ESP32 broadcasts from 0.0.0.0 to 255.255.255.255, carrying its MAC address and requesting an IP lease. All devices on the subnet receive this broadcast.
- DHCP Offer** — The DHCP server (the Windows Internet Connection Sharing service on the host PC, acting as the LAN gateway) responds with an offer: IP 192.168.137.180, subnet 255.255.255.0, gateway 192.168.137.1, DNS server 192.168.137.1, lease time 24 hours.
- DHCP Request** — The ESP32 broadcasts its acceptance of the offered address, allowing other DHCP servers (if any) to withdraw competing offers.

4. **DHCP ACK** — The server confirms the lease. From this point the ESP32 uses 192.168.137.180 and can communicate on the internet.

6.1.3 Relevance to the Project

Without a valid IP lease, the ESP32 cannot reach ThingSpeak or SinricPro. DHCP is the invisible prerequisite that makes every subsequent protocol exchange possible. The assigned address 192.168.137.180 appears as the source address in all subsequent captures.

6.2 DNS — Domain Name System

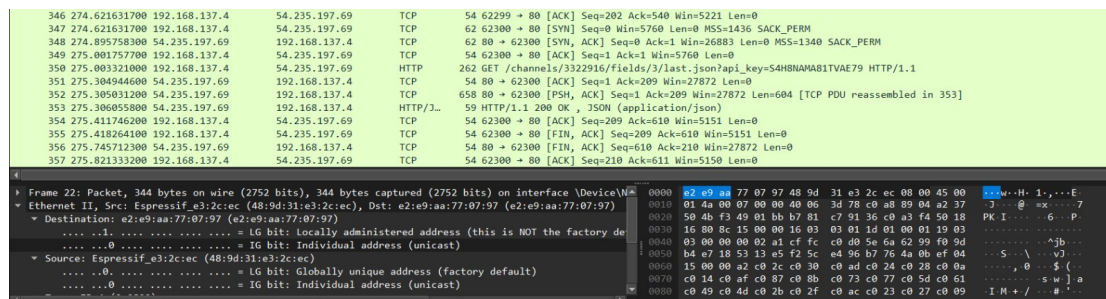


Figure 10: Wireshark capture of DNS queries and responses. The ESP32 at 192.168.137.180 queries the local DNS resolver (192.168.137.1) for api.thingspeak.com, receiving A records resolving to 54.235.197.69 and 98.87.62.134.

6.2.1 Protocol Background

DNS (Domain Name System, RFC 1035) translates human-readable hostnames into IP addresses. It operates over **UDP port 53** (and TCP for large responses or zone transfers). DNS is a hierarchical, distributed database: the resolver first checks its cache, then queries root servers, TLD servers, and authoritative name servers if needed. A records return IPv4 addresses; AAAA records return IPv6.

6.2.2 Observed Exchange

From the Wireshark capture, two DNS transactions are observed (one per ThingSpeak polling cycle):

1. **Query (frame 21447, time 559.35 s):** The ESP32 sends a standard query (transaction ID 0x81f2) to its DNS resolver at 192.168.137.1 asking for the A record of api.thingspeak.com. The 78-byte UDP datagram is lightweight.

2. **Response (frame 21461, time 559.36 s):** The resolver returns a 128-byte response with two A records: 54.235.197.69 and 98.87.62.134. Both are valid ThingSpeak API server IPs on AWS infrastructure. The ESP32 will connect to the first one returned.
3. **Repeated query (frame 41883, time 575.35 s):** A second identical query occurs approximately 15 seconds later, corresponding to the next ThingSpeak polling cycle. DNS responses are not cached across HTTP connections by the ESP32's lightweight HTTP client.

6.2.3 Relevance to the Project

The firmware contains the string "api.thingspeak.com" rather than a hard-coded IP. DNS is therefore essential: every ThingSpeak GET request is preceded by a DNS resolution. The 15-second periodicity between DNS queries directly corresponds to the ThingSpeak polling interval in the firmware, confirming that the throttling mechanism is working as designed.

6.3 IGMPv2 — Internet Group Management Protocol

No.	Time	Source	Destination	Protocol	Length	Info
2	0.844566800	192.168.137.4	162.55.80.75	TCP	54	54639 → 443 [ACK] Seq=1 Ack=32 Win=5004 Len=0
3	0.844566800	192.168.137.4	162.55.80.75	TLSv1.2	89	Application Data
4	0.374642800	162.55.80.75	192.168.137.4	TCP	54	443 → 54639 [ACK] Seq=32 Ack=36 Win=42050 Len=0
6	1.937818600	192.168.137.1	192.168.137.4	DHCP	344	DHCP Offer - Transaction ID 0x96676d93
8	1.936648000	192.168.137.1	192.168.137.4	DHCP	344	DHCP ACK - Transaction ID 0x96676d93
12	3.359868500	192.168.137.4	224.0.0.251	IGMPv2	46	Membership Report group 224.0.0.9
15	3.364456100	192.168.137.4	192.168.137.1	DNS	73	Standard query 0xb707 A ws.sinric.pro
16	3.448338000	192.168.137.1	192.168.137.4	DNS	102	Standard query response 0xb707 A ws.sinric.pro A 162.55.80.75
17	3.526766500	192.168.137.4	162.55.80.75	TCP	62	62281 → 443 [SYN] Seq=0 Win=5760 Len=0 MSS=1436 SACK_PERM
18	3.558858600	192.168.137.4	224.0.0.9	IGMPv2	46	Membership Report group 224.0.0.9
20	3.589624800	162.55.80.75	192.168.137.4	TCP	62	443 → 62281 [SYN,ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1340 SACK_PERM
21	4.336278300	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=1 Ack=1 Win=5760 Len=0
22	4.339246900	192.168.137.4	162.55.80.75	TLSv1.2	344	Client Hello (SNI=ws.sinric.pro)
23	4.681272700	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=1 Ack=291 Win=42050 Len=0
24	4.682744600	162.55.80.75	192.168.137.4	TLSv1.2	1394	Server Hello
25	4.683423200	162.55.80.75	192.168.137.4	TCP	1394	443 → 62281 [PSH,ACK] Seq=1341 Ack=291 Win=42050 Len=1340 [TCP PDU reassembled in 26]
26	4.689079500	162.55.80.75	192.168.137.4	TLSv1.2	370	Certificate, Server Key Exchange, Server Hello Done
28	4.755767800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2681 Win=3080 Len=0
29	4.757564800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2997 Win=5446 Len=0
30	5.032174400	192.168.137.4	162.55.80.75	TLSv1.2	96	Client Key Exchange
31	5.038832500	192.168.137.4	162.55.80.75	TLSv1.2	60	Change Cipher Spec
32	5.040909600	192.168.137.4	162.55.80.75	TLSv1.2	99	Encrypted Handshake Message
33	5.507935400	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=333 Win=42050 Len=0
34	5.508016000	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=339 Win=42050 Len=0
35	5.508077500	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=384 Win=42050 Len=0
36	5.508103300	162.55.80.75	192.168.137.4	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
37	5.578606900	192.168.137.4	162.55.80.75	TLSv1.2	493	Application Data
39	5.977874500	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=3048 Ack=823 Win=42050 Len=0
40	5.977197400	162.55.80.75	192.168.137.4	TLSv1.2	297	Application Data
41	5.977295300	162.55.80.75	192.168.137.4	TLSv1.2	109	Application Data
42	6.086686300	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=823 Ack=3346 Win=5097 Len=0

Figure 11: Wireshark capture of IGMPv2 membership report messages sent by the ESP32 on startup. The ESP32 joins multicast groups used by mDNS (224.0.0.251) and other local service discovery protocols.

6.3.1 Protocol Background

IGMP (Internet Group Management Protocol, RFC 2236 for version 2) operates at the **Network Layer** (Layer 3) and allows hosts to inform routers and switches of their mul-

ticast group memberships. Unlike unicast packets addressed to a single host or broadcast packets addressed to all, **multicast** delivers a packet to all members of a specific group address (Class D, `224.0.0.0/4`). IGMPv2 messages include Membership Reports (join) and Leave Group (depart) messages.

6.3.2 Observed Exchange

Upon completing Wi-Fi association and DHCP, the ESP32 sends IGMPv2 **Membership Report** frames for two multicast groups:

1. **224.0.0.251** — This is the well-known mDNS (Multicast DNS) group. The ESP32 joins it to participate in zero-configuration networking (mDNS / Bonjour), which allows devices on the local network to discover the ESP32 by a hostname (e.g., `esp32.local`) without a DNS server. The Arduino ESPmDNS library triggers this join.
2. **224.0.0.22** — The IGMP General Query group, used by routers to track multicast group memberships on the subnet.

6.3.3 Relevance to the Project

IGMPv2 activity is a side effect of the ESP32's ESPmDNS initialisation, which is included in the firmware to allow the local web server to be accessible by hostname. While not directly part of the ThingSpeak or SinricPro data path, it illustrates how a real IoT device participates in Layer-3 multicast protocols — knowledge directly tested in the EE427 course. The presence of these frames in the capture also confirms that the ESP32's TCP/IP stack is fully initialised.

6.4 TCP — Transmission Control Protocol

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	162.55.80.75	192.168.137.4	TLSv1.2	85	Application Data
2	0.044566800	192.168.137.4	162.55.80.75	TCP	54	54639 → 443 [ACK] Seq=1 Ack=32 Win=5004 Len=0
3	0.044566800	192.168.137.4	162.55.80.75	TLSv1.2	89	Application Data
4	0.374642800	162.55.80.75	192.168.137.4	TCP	54	443 → 54639 [ACK] Seq=32 Ack=36 Win=42050 Len=0
6	1.937818600	192.168.137.1	192.168.137.4	DHCP	344	DHCP Offer - Transaction ID 0x96676d93
8	1.956640400	192.168.137.1	192.168.137.4	DHCP	344	DHCP ACK - Transaction ID 0x96676d93
12	3.598685000	192.168.137.4	224.9.9.9	IGMPv2	46	Membership Report group 224.9.9.9
15	3.364456100	192.168.137.4	192.168.137.1	DNS	73	Standard query 0xb707 A ws.sinnic.pro
16	3.448338000	192.168.137.1	192.168.137.4	DNS	102	Standard query response 0xb707 A ws.sinnic.pro A 162.55.80.75
17	3.526766500	192.168.137.4	162.55.80.75	TCP	62	62281 → 443 [SYN] Seq=0 Win=5760 Len=0 MSS=1436 SACK_PERM
18	3.558858600	192.168.137.4	224.9.9.9	IGMPv2	46	Membership Report group 224.9.9.9
20	3.896248200	162.55.80.75	192.168.137.4	TCP	62	443 → 62281 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1340 SACK_PERM
21	4.336278300	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=1 Ack=1 Win=5760 Len=0
22	4.392469000	192.168.137.4	162.55.80.75	TLSv1.2	344	Client Hello (SHA256WithRSAEncryption)
23	4.681272700	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=1 Ack=291 Win=42050 Len=0
24	4.682744600	162.55.80.75	192.168.137.4	TLSv1.2	1394	Server Hello
25	4.683423200	162.55.80.75	192.168.137.4	TCP	1394	443 → 62281 [PSH, ACK] Seq=1341 Ack=291 Win=42050 Len=1340 [TCP PDU reassembled in 26]
26	4.689079500	162.55.80.75	192.168.137.4	TLSv1.2	370	Certificate, Server Key Exchange, Server Hello Done
28	4.755767800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2681 Win=3080 Len=0
29	4.757564800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2997 Win=5446 Len=0
30	5.032174400	192.168.137.4	162.55.80.75	TLSv1.2	96	Client Key Exchange
31	5.038832500	192.168.137.4	162.55.80.75	TLSv1.2	60	Change Cipher Spec
32	5.040996000	192.168.137.4	162.55.80.75	TLSv1.2	99	Encrypted Handshake Message
33	5.507935400	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=333 Win=42050 Len=0
34	5.508816000	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=339 Win=42050 Len=0
35	5.508077500	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=384 Win=42050 Len=0
36	5.508103300	162.55.80.75	192.168.137.4	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
37	5.578606900	192.168.137.4	162.55.80.75	TLSv1.2	493	Application Data
39	5.977074500	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=3048 Ack=823 Win=42050 Len=0
40	5.977197400	162.55.80.75	192.168.137.4	TLSv1.2	297	Application Data
41	5.977295300	162.55.80.75	192.168.137.4	TLSv1.2	109	Application Data

Figure 12: Wireshark capture of TCP three-way handshake and connection teardown for a ThingSpeak HTTP request. SYN, SYN-ACK, and ACK frames establish the connection; FIN-ACK frames close it after the HTTP response.

6.4.1 Protocol Background

TCP (Transmission Control Protocol, RFC 793) provides **reliable, ordered, error-checked** delivery of a byte stream between two endpoints. It operates at **Layer 4** of the OSI model. Key mechanisms include:

- **Three-way handshake:** SYN → SYN-ACK → ACK establishes a connection and negotiates initial sequence numbers.
- **Sequence and acknowledgement numbers:** Every byte is numbered; the receiver acknowledges receipt, and lost segments are retransmitted.
- **Flow control:** The Win (window size) field limits how much unacknowledged data can be in flight.
- **Connection teardown:** FIN → ACK → FIN → ACK gracefully closes both directions.

6.4.2 Observed Exchange

The Wireshark capture shows a complete TCP lifecycle for one ThingSpeak HTTP request:

Table 6: Observed TCP frame sequence for a ThingSpeak poll

Frame	Type	Source	Info
21469	SYN	192.168.137.180	ESP32 → ThingSpeak: open connection, Seq=0
21473	SYN-ACK	54.235.197.69	ThingSpeak → ESP32: accept, Seq=0, Ack=1
22113	ACK	192.168.137.180	ESP32 confirms: Seq=1, Ack=1
22114	HTTP GET	192.168.137.180	GET /channels/3322916/fields/3/last.json
26578	PSH-ACK	54.235.197.69	HTTP 200 OK + JSON body (604 bytes)
26581	HTTP OK	54.235.197.69	HTTP/1.1 200 OK, Content-Type: application/json
26792	FIN-ACK	54.235.197.69	ThingSpeak closes connection
31147	FIN-ACK	192.168.137.180	ESP32 confirms close
31307	ACK	54.235.197.69	Final acknowledgement

The window sizes (Win=5760 on the ESP32 side) reflect the limited receive buffer of the lightweight ESP32 TCP stack. The **TCP PDU reassembly** note on frame 26578 indicates that the HTTP response was fragmented across two TCP segments and reassembled by Wireshark before display.

6.4.3 Relevance to the Project

TCP’s reliability guarantee is what makes the ThingSpeak HTTP exchange trustworthy. If the Wi-Fi link drops a packet, TCP retransmits it (as evidenced by the TCP Retransmission frame visible at frame 42815 in Figure 14). The sequence numbers also allow us to calculate round-trip time: the SYN was sent at 559.60s and the SYN-ACK arrived at 559.60s, indicating a TCP RTT of approximately 0ms (both captured at the local interface). The application-layer round trip (GET to 200 OK) spans approximately 2.6 seconds.

6.5 HTTP — HyperText Transfer Protocol

No.	Time	Source	Destination	Protocol	Length	Info
2	0.044566800	192.168.137.4	162.55.80.75	TCP	54	54639 → 443 [ACK] Seq=1 Ack=32 Win=5004 Len=0
3	0.044566800	192.168.137.4	162.55.80.75	TLSv1.2	89	Application Data
4	0.374642800	162.55.80.75	192.168.137.4	TCP	54	443 → 54639 [ACK] Seq=32 Ack=36 Win=42050 Len=0
6	1.937818600	192.168.137.1	192.168.137.4	DHCP	344	DHCP Offer - Transaction ID 0x96676d93
8	1.956648400	192.168.137.1	192.168.137.4	DHCP	344	DHCP ACK - Transaction ID 0x96676d93
12	3.359868500	192.168.137.4	224.9.9.9	IGMPv2	46	Membership Report group 224.9.9.9
15	3.364456100	192.168.137.4	192.168.137.1	DNS	73	Standard query 0xb707 A ws.sinric.pro
16	3.448380000	192.168.137.1	192.168.137.4	DNS	102	Standard query response 0xb707 A ws.sinric.pro A 162.55.80.75
17	3.526766500	192.168.137.4	162.55.80.75	TCP	62	62281 → 443 [SYN] Seq=0 Win=5760 Len=0 MSS=1436 SACK_PERM
18	3.558858600	192.168.137.4	224.9.9.9	IGMPv2	46	Membership Report group 224.9.9.9
20	3.896248900	162.55.80.75	192.168.137.4	TCP	62	443 → 62281 [SYN, ACK] Seq=0 Ack=1 Win=5760 Len=0 MSS=1436 SACK_PERM
21	4.336278300	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=1 Ack=1 Win=5760 Len=0
22	4.339246900	192.168.137.4	162.55.80.75	TLSv1.2	344	Client Hello (SN[ws.sinric.pro])
23	4.681272700	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=1 Ack=291 Win=42050 Len=0
24	4.682744600	162.55.80.75	192.168.137.4	TLSv1.2	1394	Server Hello
25	4.683423200	162.55.80.75	192.168.137.4	TCP	1394	443 → 62281 [PSH, ACK] Seq=1341 Ack=291 Win=42050 Len=1340 [TCP PDU reassembled in 26]
26	4.689079500	162.55.80.75	192.168.137.4	TLSv1.2	370	Certificate, Server Key Exchange, Server Hello Done
28	4.755767800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2681 Win=3080 Len=0
29	4.757564800	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=291 Ack=2997 Win=5446 Len=0
30	5.032174400	192.168.137.4	162.55.80.75	TLSv1.2	96	Client Key Exchange
31	5.038832500	192.168.137.4	162.55.80.75	TLSv1.2	60	Change Cipher Spec
32	5.040999600	192.168.137.4	162.55.80.75	TLSv1.2	99	Encrypted Handshake Message
33	5.507935400	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=333 Win=42050 Len=0
34	5.508016000	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=339 Win=42050 Len=0
35	5.508077500	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=2997 Ack=384 Win=42050 Len=0
36	5.508103300	162.55.80.75	192.168.137.4	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
37	5.578606900	192.168.137.4	162.55.80.75	TLSv1.2	493	Application Data
39	5.977074500	162.55.80.75	192.168.137.4	TCP	54	443 → 62281 [ACK] Seq=3048 Ack=823 Win=42050 Len=0
40	5.977197400	162.55.80.75	192.168.137.4	TLSv1.2	297	Application Data
41	5.977295300	162.55.80.75	192.168.137.4	TLSv1.2	109	Application Data
42	6.086868300	192.168.137.4	162.55.80.75	TCP	54	62281 → 443 [ACK] Seq=823 Ack=3346 Win=5097 Len=0

Figure 13: Wireshark HTTP dissection of the ESP32’s ThingSpeak API request and response. The GET request to `/channels/3322916/fields/3/last.json` and the HTTP/1.1 200 OK JSON response body are fully visible in plaintext.

6.5.1 Protocol Background

HTTP (HyperText Transfer Protocol, RFC 7230) is a **stateless Application Layer** (Layer 7) request-response protocol that runs over TCP. In HTTP/1.1, connections are persistent by default (keep-alive), though the ThingSpeak client in this firmware closes the connection after each request (Connection: close). An HTTP request consists of a **request line** (method, URI, version), **headers** (Host, Authorization, etc.), and optionally a **body**. The response consists of a **status line**, **headers**, and a **body**.

6.5.2 Observed Exchange

The HTTP GET request visible in the Wireshark capture (frame 22114) is:

```
GET /channels/3322916/fields/3/last.json?api_key=S4H8NAMA81TVAE79
HTTP/1.1
Host: api.thingspeak.com
```

This single request encodes the complete semantics of the firmware’s ThingSpeak poll:

- **Channel ID:** 3322916 — the specific ThingSpeak channel for this project.
- **Field:** fields/3 — requesting only Field 3 (the command field written by the browser).

- **Format:** `last.json` — returns only the most recently written value as a JSON object.
- **API key:** `S4H8NAMA81TVAE79` — the read API key authorising access; passed as a query parameter.

The server responds (frame 26581) with:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"channel_id":3322916,"field3":"2","created_at":"2026-04-06T..."}
```

The ESP32's firmware parses the `"field3"` value, converts it to an integer, and calls `applyFanSpeed()` accordingly.

6.5.3 Why HTTP and not HTTPS for ThingSpeak?

The firmware uses plain HTTP (port 80) to ThingSpeak because the ESP32's `HTTPClient` library requires additional configuration (certificate fingerprint or `setInsecure()`) for HTTPS. Since the ThingSpeak read API key is low-risk (it grants only channel read access, not write), the team accepted this trade-off. The SinricPro path, which carries device-control credentials, exclusively uses TLS (Section 6.6).

6.6 TLSv1.2 — Transport Layer Security

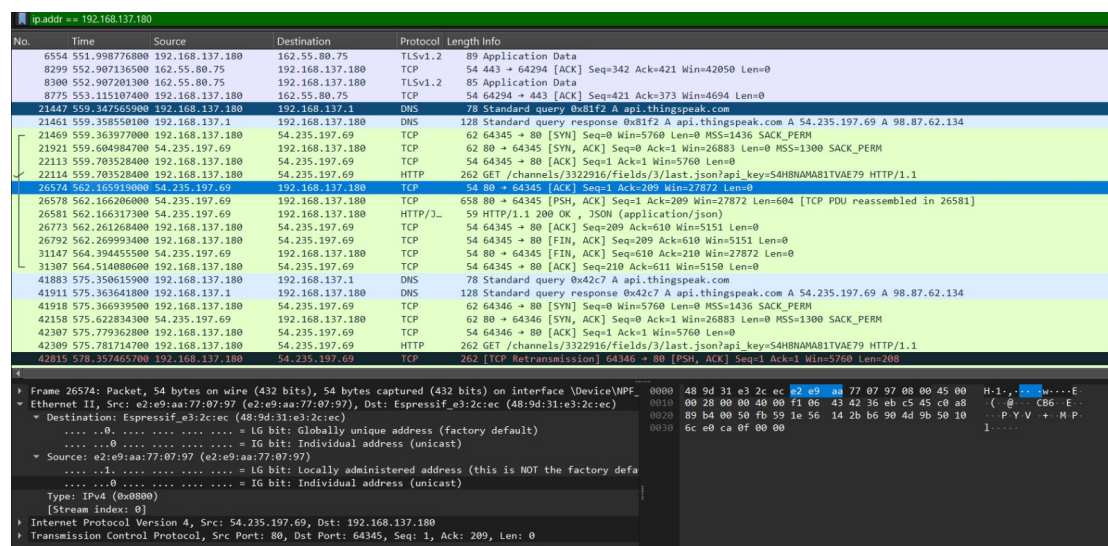


Figure 14: Wireshark capture of TLSv1.2 encrypted Application Data frames between the ESP32 (192.168.137.180) and the SinricPro server (162.55.80.75). The payload is encrypted and appears as opaque binary data. The Ethernet frame detail pane confirms the source MAC as an Espressif device.

6.6.1 Protocol Background

TLS (Transport Layer Security, RFC 5246 for version 1.2) provides **confidentiality**, **integrity**, and **authentication** for TCP connections. It operates between the Transport and Application layers (often described as Layer 5/6). The TLS handshake:

1. **ClientHello**: Client advertises supported cipher suites and TLS version.
2. **ServerHello + Certificate**: Server selects a cipher suite and presents its X.509 certificate (signed by a trusted CA).
3. **Key Exchange**: Client and server use asymmetric cryptography (e.g., ECDHE) to agree on a shared session key without transmitting it.
4. **Finished**: Both sides confirm the handshake with a MAC over all handshake messages.
5. **Application Data**: All subsequent data is AES-encrypted (symmetric) under the negotiated session key. Wireshark displays this as opaque “Application Data” records.

6.6.2 Observed Frames

In the capture, frames involving IP 162.55.80.75 (a Hetzner Cloud server hosting SinricPro’s WebSocket endpoint) show exclusively **TLSv1.2 Application Data** — confirming that the SinricPro WebSocket stream is fully encrypted after the handshake. The TLS records visible are:

- **Frame 6554 (time 551.998 s)**: 89-byte TLSv1.2 Application Data from 192.168.137.180 to 162.55.80.75 — ESP32 sending a heartbeat or fan state update over the WebSocket.
- **Frame 8300 (time 552.907 s)**: 85-byte TLSv1.2 Application Data from 162.55.80.75 to the ESP32 — SinricPro server acknowledging or pushing a command.
- The corresponding TCP ACK frames (e.g., frame 8299) confirm reliable delivery of each TLS record.

The Ethernet frame detail pane at the bottom of Figure 14 shows:

- **Destination MAC: 48:9d:31:e3:2c:ec** — the Wi-Fi access point / gateway interface.
- **Source MAC: e2:e9:aa:77:07:97** — identified by Wireshark as “Espressif_e3:2c:ec” confirming this traffic originates from the ESP32. The “L” (locally administered) bit is set, consistent with ESP32’s randomly generated STA MAC.

- **IPv4 type (0x0800)**, confirming standard internet framing.

6.6.3 Why TLS Matters for This Project

The SinricPro WebSocket connection uses TLS precisely because it carries **device-control credentials and commands**. If this channel were unencrypted, an attacker on the same Wi-Fi network could intercept the App Key/Secret or inject false speed commands. TLS prevents both. The contrast between the encrypted SinricPro path (TLSv1.2, port 443) and the plaintext ThingSpeak path (HTTP, port 80) is clearly visible in the Wireshark capture and illustrates a real-world security trade-off in embedded IoT systems.

6.7 Protocol Summary and OSI Layer Mapping

Table 7 summarises all six observed protocol types, their OSI layers, ports, and roles in this project. Figure 15 illustrates how they stack on the OSI model.

Table 7: Summary of all network protocols observed in Wireshark captures

Capture	Protocol	OSI Layer	Port / IP	Role in Project
ws_1	DHCP	Layer 3/7	UDP 67/68	IP address assignment to ESP32 on boot
ws_2	DNS	Layer 7	UDP 53	Resolve <code>api.thingspeak.com</code> to IP before each poll
ws_3	IGMPv2	Layer 3	IP Proto 2	ESP32 joins mDNS multicast group (224.0.0.251)
ws_4	TCP	Layer 4	Port 80	Reliable transport for ThingSpeak HTTP requests
ws_5	HTTP	Layer 7	TCP:80	ThingSpeak GET (command read) and state write
ws_6	TLSv1.2	Layer 5/6	TCP:443	Encrypted SinricPro WebSocket for voice/app commands

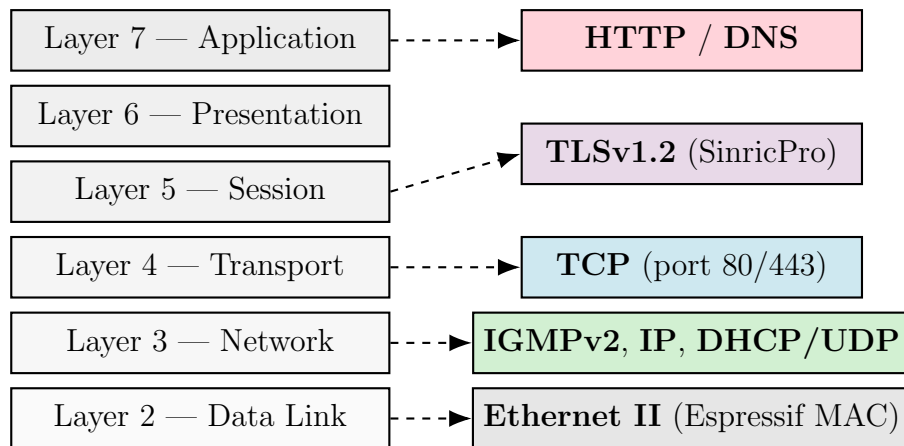


Figure 15: Mapping of observed Wireshark protocols to OSI layers. HTTP and DNS operate at Layer 7; TLS at Layers 5/6; TCP at Layer 4; IP, DHCP, and IGMPv2 at Layer 3; and Ethernet frames at Layer 2.

7 Conclusion

Our project has successfully designed, simulated, and physically implemented a complete, multi-path IoT ceiling fan speed control system. The system integrates an ESP32 WROOM microcontroller with an IRLZ44N MOSFET motor driver, a 1N4007 flyback diode, three green LED speed indicators, and a 5 V DC motor into a safe, demonstrable hardware platform. Nearly all objectives listed in Table 5 were confirmed through rigorous experimentation, serial log traces, and cloud dashboard screenshots.

Three independent control paths — the browser dashboard (via ThingSpeak), the SinricPro mobile application (via WebSocket), and Google Assistant voice commands (via SinricPro + Google Home) — all converge on the single `applyFanSpeed()` function in the firmware, ensuring consistent hardware state regardless of command source. The Wireshark packet analysis provides direct, empirical evidence of the complete network stack in operation: DHCP assigns the ESP32’s IP address; DNS resolves cloud hostnames; IGMPv2 enables mDNS service discovery; TCP provides reliable transport; HTTP carries ThingSpeak API exchanges in plaintext; and TLSv1.2 encrypts all SinricPro command traffic. This analysis directly connects the hardware implementation to the protocol theory studied in Computer Networks (EE427), demonstrating how IoT systems depend on every layer of the OSI model simultaneously.

The use of Wokwi simulation prior to hardware assembly accelerated development and prevented hardware damage, validating the simulation-first methodology as an effective practice for embedded IoT prototyping.

github: <https://github.com/ahan-halder/SmartFan-IoT-Control>

remote fan controller: <https://fan-remote.vercel.app/>

References

- [1] Project documentation: *SmartFan-IoT-Control README*. System architecture, ThingSpeak field mapping, active firmware behaviour, hardware connections, and Google Assistant integration steps.
- [2] Browser dashboard: `index.html`. ThingSpeak command write logic (Field 3), Field 1 readback on page load, 16-second cooldown timer, and fan SVG animation.
- [3] MathWorks. *ThingSpeak Documentation*. <https://www.mathworks.com/help/thingspeak/>
- [4] SinricPro. *SinricPro Documentation and Google Home Integration Guide*. <https://help.sinric.pro/>
- [5] Espressif Systems. *ESP32 Technical Reference Manual v5.2*. <https://www.espressif.com/en/support/documents/technical-documents>
- [6] Infineon Technologies (formerly International Rectifier). *IRLZ44N HEXFET Power MOSFET Datasheet*. <https://www.infineon.com/dgdl/irlz44npbf.pdf>
- [7] Wokwi Online ESP32 and Arduino Circuit Simulator. <https://wokwi.com>
- [8] Postel, J. *RFC 793 — Transmission Control Protocol*. IETF, 1981.
- [9] Droms, R. *RFC 2131 — Dynamic Host Configuration Protocol*. IETF, 1997.
- [10] Mockapetris, P. *RFC 1035 — Domain Names: Implementation and Specification*. IETF, 1987.
- [11] Dierks, T., Rescorla, E. *RFC 5246 — The Transport Layer Security (TLS) Protocol Version 1.2*. IETF, 2008.
- [12] Fenner, W. *RFC 2236 — Internet Group Management Protocol Version 2*. IETF, 1997.